Guion de trabajo tutorizado

Google Cloud, servicio y pipeline mediante ejemplos

Plataformas Tecnológicas

Profesor D. Alejandro Gómez Morón
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «GCloud - TrabajoTutorizado.pdf» (Campus Virtual UPO).

Objetivos

- Saber qué tipo de servicios ofrece Google Cloud Platform (GCP).
- Ser capaz de diseñar soluciones Big Data en GCP
- Saber cómo utilizar la documentación ofrecida por GCP
- Conocimiento sobre cómo sacar el mayor partido de GCP para ti y para tu empresa

Actividades

Las siguientes preguntas pueden ser cosas que hayamos visto en clase explícitamente o no. El objetivo es que el alumno aprenda a diferenciar entre los tipos de productos y servicios que GCP ofrece. La actividad está dividida en dos bloques: preguntas y ejercicios.

Las preguntas deberá contestarlas razonadas con links a cualquier documentación usada y útil y, para los problemas, vendrá en cada uno lo que se requiere.

Preguntas

Pregunta 1 (60 minutos - 1,5 puntos).- Si tengo que desplegar una funcionalidad que sólo hace una transformación rápida en tiempo real, ¿cuál es la ventaja de usar una Cloud Function vs utilizar mi instancia de Compute Engine?

Pregunta 2 (75 minutos - 2 puntos).- Si nuestra empresa sabe que va a recibir en tiempo real una cantidad ingente de datos, ¿qué mecanismo de ingesta debería poner para recibir los datos? ¿Por qué?

Pregunta 3 (60 minutos - 1,5 puntos).- Si vamos a desplegar código en Cloud Function escrito en Node 16, ¿qué ocurriría según la documentación? ¿podríamos y funciona? ¿no podríamos y no funciona? ¿podríamos y no funciona? Detalle la respuesta y argumento acorde a la documentación de GCP.

Ejercicios prácticos

Ejercicio 1 (15 minutos - 0,5 puntos).- Crearse una cuenta de GCP.

Ejercicio 2 (80 minutos - 3 puntos).- Cree una pipeline utilizando Cloud Run Functions donde:

- Tenga un bucket en la región que desee.
- Que la Cloud Function de segunda generación se ejecute cuando se suba un fichero (https://docs.cloud.google.com/functions/docs/samples/functions-cloudevent-stor age?hl=es)
- Que la Cloud Function borre el fichero que se ha subido

Ejercicio 3 (80 minutos - 1,5 puntos) . Usando el video que se ha subido al Aula Virtual, importe los datos de un dataset que elija (no use NYC taxis) y haced lo mismo: usad una parte de los datos y cread un modelo automático con BigQuery. Aporte evidencias de:

- Dataset elegido.
- Bucket creado.
- Cómo crea conjuntos de entrenamientos.
- Resultado del modelo.

• Un párrafo explicando qué entiende por el output que obtiene del modelo automático generado por BigQuery.

IMPORTANTE: Recuerde eliminar los recursos creados para la práctica

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 19 de enero de 2025.

Bibliografía

1. Google Cloud: https://cloud.google.com/docs/

2. Cloud Functions: https://cloud.google.com/functions/docs/

3. Compute Engine: https://cloud.google.com/compute/docs/

4. Free tier: https://cloud.google.com/free/

5. Ejemplos de CloudFunctions: https://cloud.google.com/functions/docs/samples

RESOLUCIÓN

Pregunta 1. Cloud Function frente a una instancia de Compute Engine para una transformación rápida en tiempo real

Para una funcionalidad que solo hace **una transformación breve y puntual** (validar un JSON, redimensionar una imagen, enriquecer un evento y reenviarlo), Cloud Run Functions es la opción natural: es un modelo **dirigido por eventos** en el que solo se escribe la función y la plataforma se encarga del resto.

Criterio	Cloud Run Function (2ª gen.)	Compute Engine
Unidad de trabajo	Una función que se ejecuta por evento o petición HTTP.	Una máquina virtual que hay que aprovisionar, arrancar y mantener.
Facturación	Por invocación y por tiempo de cómputo real (100 ms), con capa gratuita mensual. **Escala a cero**: si no hay eventos, no se paga.	Por tiempo de instancia encendida, se use o no. Una VM ociosa sigue facturando.
Escalado	Automático e instantáneo, de 0 a miles de instancias concurrentes; se puede acotar con `--max-instances`.	Manual, o mediante grupos de instancias gestionados y balanceador, que hay que configurar y ajustar.
Operación	Sin sistema operativo que parchear: Google gestiona runtime, seguridad y disponibilidad.	El equipo es responsable del SO, las actualizaciones, el *hardening*, los logs y la monitorización.
Integración con eventos	Nativa vía Eventarc: Cloud Storage, Pub/Sub, Firestore, Audit Logs… sin escribir el *listener*.	Hay que programar el consumidor, gestionar reintentos y mantenerlo vivo.
Tiempo hasta producción	Un `gcloud functions deploy` y está publicado.	Imagen, red, reglas de cortafuegos, despliegue del código, servicio systemd…

La ventaja principal es económica y operativa: se paga solo por el tiempo de ejecución real y desaparece toda la carga de administración. Un tráfico irregular —el caso típico de una transformación disparada por subidas de ficheros— deja la VM infrautilizada la mayor parte del tiempo, mientras que la función no cuesta nada cuando no hay eventos y absorbe los picos sin intervención.

Cuándo no compensa: procesos de larga duración o que superen el tiempo máximo de ejecución, cargas sostenidas y predecibles (a alto volumen constante la VM reservada sale más barata), necesidades de GPU, de estado en disco, de software con licencia o de un control fino del sistema operativo, y escenarios donde el *arranque en frío* sea inaceptable. Entre ambos extremos está **Cloud Run**, que mantiene el modelo *serverless* pero permite contenedores propios y peticiones más largas.

Referencias

Cloud Run Functions: https://cloud.google.com/functions/docs/

Compute Engine: https://cloud.google.com/compute/docs/

Comparativa de opciones de cómputo: https://cloud.google.com/hosting-options

Pregunta 2. Mecanismo de ingesta para un volumen ingente de datos en tiempo real

El mecanismo adecuado es **Cloud Pub/Sub** como capa de ingesta, seguido de **Dataflow** (Apache Beam) para el procesado en *streaming* y de BigQuery o Cloud Storage como destino.

Por qué Pub/Sub. Es un bus de mensajería *serverless*, global y con escalado automático, que actúa como **amortiguador entre productores y consumidores**:

- **Desacoplamiento.** Los productores publican sin conocer a los consumidores. Si el sistema de proceso se cae o se ralentiza, los mensajes se retienen (hasta 7 días de forma predeterminada, 31 configurable) y no se pierde nada: el consumidor los recupera cuando vuelve.
- **Absorción de picos.** Un pico de tráfico llena la cola en lugar de tumbar el sistema de destino. Sin esta capa, una ráfaga que supere la capacidad de escritura de la base de datos provoca pérdida de eventos.
- **Escalado sin aprovisionamiento.** Admite millones de mensajes por segundo sin que haya que dimensionar particiones ni brokers, a diferencia de un Kafka autogestionado.
- **Entrega al menos una vez**, con confirmaciones explícitas, reintentos y *dead-letter topics*; y **Pub/Sub Lite** o el modo *exactly-once* cuando el caso lo exige.
- **Fan-out.** Varias suscripciones sobre el mismo topic permiten alimentar a la vez el flujo en tiempo real y un volcado al lago para reproceso histórico.

Arquitectura completa recomendada: dispositivos o aplicaciones → *(Cloud IoT/gateway si son dispositivos)* → **Pub/Sub** → **Dataflow** en modo *streaming* (ventanas, agregaciones, deduplicación, datos tardíos) → **BigQuery** para analítica y **Cloud Storage** para el histórico en bruto → Looker Studio para visualización. Para cargas simples que no requieren transformación, **BigQuery Storage Write API** o las suscripciones de Pub/Sub a BigQuery escriben directamente sin pasar por Dataflow.

Alternativas y cuándo elegirlas. *Managed Service for Apache Kafka* o *Confluent Cloud* si ya existe ecosistema Kafka o se necesita retención larga y relectura de offsets; **Datastream** para replicación de bases de datos en tiempo real (CDC); **Bigtable** como destino cuando se requieren lecturas de baja latencia por clave. Lo que no debe hacerse es escribir directamente desde los productores a la base de datos analítica: sin cola intermedia no hay amortiguación, ni reintentos, ni desacoplamiento.

Referencias

Pub/Sub: https://cloud.google.com/pubsub/docs/overview

Dataflow (streaming): https://cloud.google.com/dataflow/docs/concepts/streaming-pipelines

Patrones de ingesta en GCP: https://cloud.google.com/architecture/data-lifecycle-cloud-platform

Pregunta 3. Desplegar código en Cloud Functions escrito en Node 16

Respuesta: no podríamos desplegarlo y, por tanto, no funcionaría. No es un caso de "se despliega pero falla en ejecución": la propia orden de despliegue es rechazada antes de construir nada.

Google publica un calendario de soporte de entornos de ejecución ligado al ciclo de vida de cada lenguaje. Node.js 16 llegó a su fin de vida en septiembre de 2023; en consecuencia, el entorno `nodejs16` quedó **obsoleto (deprecated)** el **30 de enero de 2024** y fue **retirado (decommissioned)** el **30 de enero de 2025**. La documentación distingue con precisión las dos fases:

- **Fase de obsolescencia:** se puede seguir desplegando y actualizando funciones, pero el entorno deja de recibir actualizaciones de seguridad y Google recomienda migrar.
- **Tras la retirada:** ya **no se pueden crear ni actualizar** funciones con ese entorno, y las funciones existentes pueden ser deshabilitadas. Solo una migración a un entorno soportado las devuelve al servicio.

La comprobación práctica en el proyecto confirma el comportamiento documentado: al intentar desplegar con `--runtime=nodejs16` la orden falla inmediatamente, y `gcloud functions runtimes list` solo ofrece `nodejs20` (ya marcado como DEPRECATED), `nodejs22` y `nodejs24` en disponibilidad general, y `nodejs26` en beta.

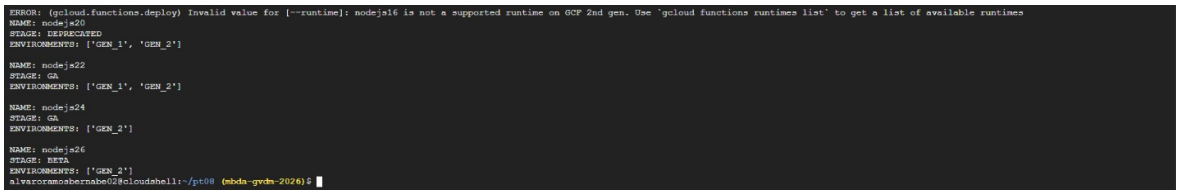


Figura 1. Rechazo del despliegue con `nodejs16` y listado de entornos de ejecución de Node.js disponibles actualmente en el proyecto.

Qué hacer: migrar el código a un entorno en GA (`nodejs22` o `nodejs24`). La migración desde Node 16 suele ser directa; conviene revisar las dependencias nativas, el uso de módulos ESM frente a CommonJS y las APIs retiradas en versiones posteriores. En funciones de 2ª generación, además, la firma del manejador se declara con el *Functions Framework* (`@google-cloud/functions-framework`), tal y como se ha hecho en el ejercicio 2.

Referencias

Entornos de ejecución de Cloud Run Functions: https://cloud.google.com/functions/docs/runtime-support

Política de obsolescencia de entornos: https://cloud.google.com/functions/docs/concepts/nodejs-runtime

Ejercicio 1. Cuenta de GCP

La cuenta de Google Cloud está creada y operativa; todo el trabajo de este documento se ha realizado sobre el proyecto **`mbda-gvdm-2026`**, en la región `europe-west1`, tal y como muestran las capturas de los ejercicios siguientes (el identificador del proyecto aparece en el indicador de Cloud Shell).

Ejercicio 2. Pipeline con Cloud Run Functions: borrar el fichero que se sube al bucket

Se crea un bucket y una **Cloud Run Function de 2ª generación** que se dispara con el evento `google.cloud.storage.object.v1.finalized` (objeto subido) y elimina el fichero recién subido.

Código de la función

La función usa el *Functions Framework* con el manejador de CloudEvents, como en el ejemplo de la documentación oficial, y borra el objeto con la librería cliente de Cloud Storage:

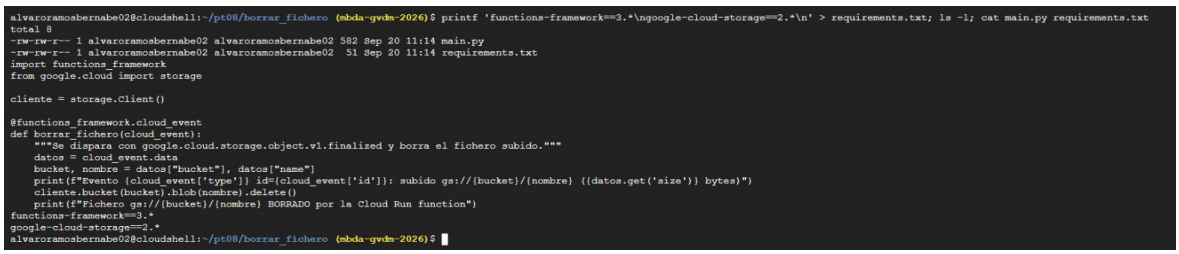


Figura 2. Código de la función `borrar_fichero` (Python 3.12, CloudEvent de Cloud Storage) y su `requirements.txt`.

Despliegue

```
gcloud storage buckets create gs://$BUCKET --location=$REGION --uniform-bucket-
level-access

gcloud functions deploy borrar-fichero --gen2 --runtime=python312 --
region=$REGION \
  --source=. --entry-point=borrar_fichero \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=$BUCKET"
```

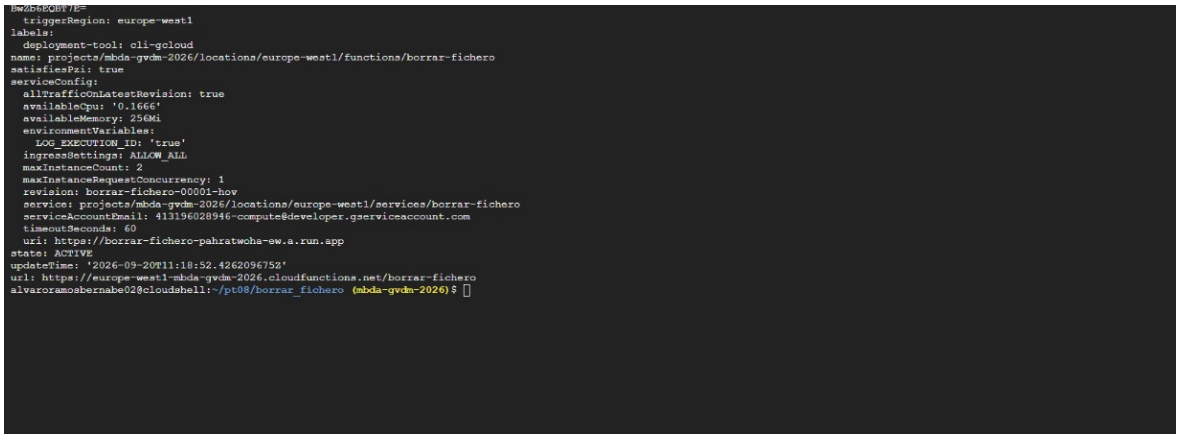


Figura 3. Despliegue de la función de 2ª generación con el disparador de Cloud Storage.

El despliegue requirió conceder a la **cuenta de servicio de Eventarc** el rol `roles/eventarc.serviceAgent`: el primer intento falló porque el agente de servicio aún no existía en el proyecto. Se creó con `gcloud beta services identity create --service=eventarc` y se repitió el despliegue. Es un paso habitual la primera vez que se usa Eventarc en un proyecto nuevo.

Prueba de funcionamiento

Se suben dos ficheros al bucket, se espera unos segundos y se vuelve a listar: el bucket queda vacío y los registros de la función muestran el evento recibido y el borrado de cada objeto.



Figura 4. Subida de `prueba_pt08.txt` y `prueba_3.txt`, listado vacío 20 s después y trazas de Cloud Logging con el evento `finalized` y el mensaje «BORRADO por la Cloud Run function» para cada fichero.

Detalle importante de diseño: la función escribe (borra) en el **mismo bucket** que dispara el evento. Esto es seguro porque el evento es `finalized` (creación de objeto) y el borrado no genera ese tipo de evento; si la función escribiera un objeto nuevo en el bucket de entrada, se produciría una **recursión infinita** de invocaciones, un error clásico en este patrón, que se evita usando un bucket de salida distinto.

Ejercicio 3. Modelo automático en BigQuery ML con un dataset propio

Dataset elegido

Se ha usado el **Bike Sharing Dataset** del UCI Machine Learning Repository (sistema Capital Bikeshare de Washington D.C., 2011-2012), fichero `hour.csv` con **17.379 registros horarios**. El objetivo es predecir la variable `cnt`: el número de bicicletas alquiladas en cada hora, a partir de la estación del año, el mes, la hora, el día de la semana, si es laborable o festivo, la situación meteorológica, la temperatura, la sensación térmica, la humedad y el

viento. Es un problema de **regresión** y no es el de los taxis de Nueva York, como exige el enunciado.

Bucket creado y carga en BigQuery

```
gcloud storage buckets create gs://$DATOS --location=$REGION --uniform-bucket-
level-access
gcloud storage cp hour.csv gs://$DATOS/bike_sharing/hour.csv

bq --location=EU mk -d pt08
bq load --autodetect --source_format=CSV pt08.bike_hour
gs://$DATOS/bike_sharing/hour.csv
```

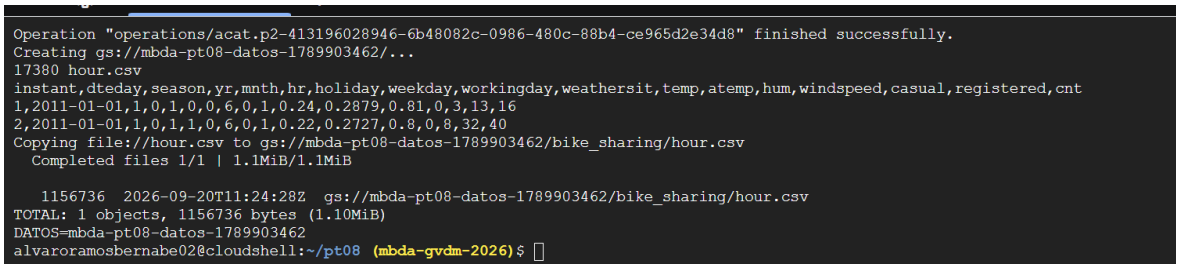


Figura 5. Bucket `mbda-pt08-datos` creado y fichero `hour.csv` subido a la ruta `bike_sharing/`.

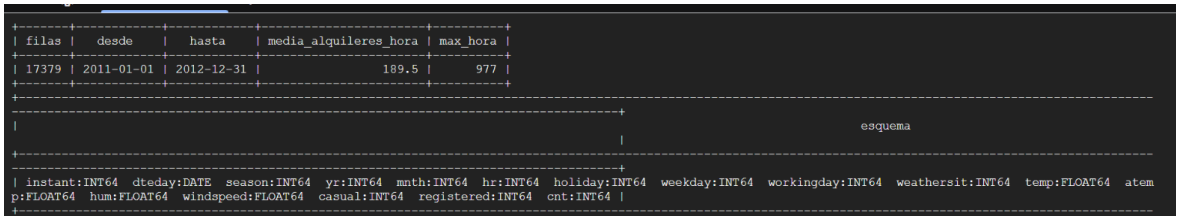


Figura 6. Tabla `pt08.bike_hour` cargada desde Cloud Storage con detección automática de esquema: 17.379 filas.

Creación de los conjuntos de entrenamiento y test

La partición se hace con SQL de forma **determinista**, aplicando una función de *hash* sobre la clave `instant` y repartiendo 80 % / 20 %. Así el reparto es reproducible: repetir la consulta devuelve exactamente los mismos conjuntos, algo que no ocurriría con `RAND()`. Se excluyen además las columnas `casual` y `registered`, cuya suma **es** la variable objetivo `cnt` y provocarían una fuga de información.

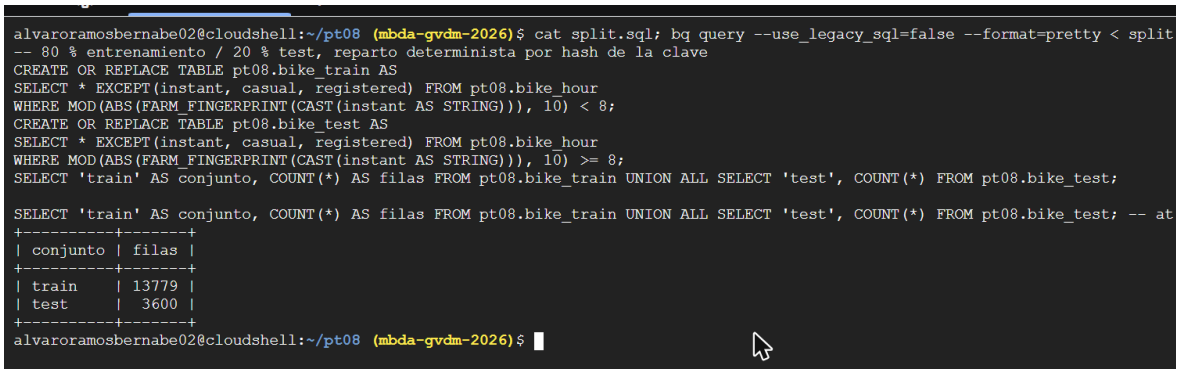


Figura 7. Creación de `bike_train` (13.779 filas) y `bike_test` (3.600 filas) mediante `FARM_FINGERPRINT` sobre la clave.

Entrenamiento del modelo automático

Se entrena un modelo `AUTOML_REGRESSOR`: BigQuery ML delega en Vertex AI AutoML Tables, que prueba y combina automáticamente distintas arquitecturas y sus hiperparámetros dentro del presupuesto indicado.

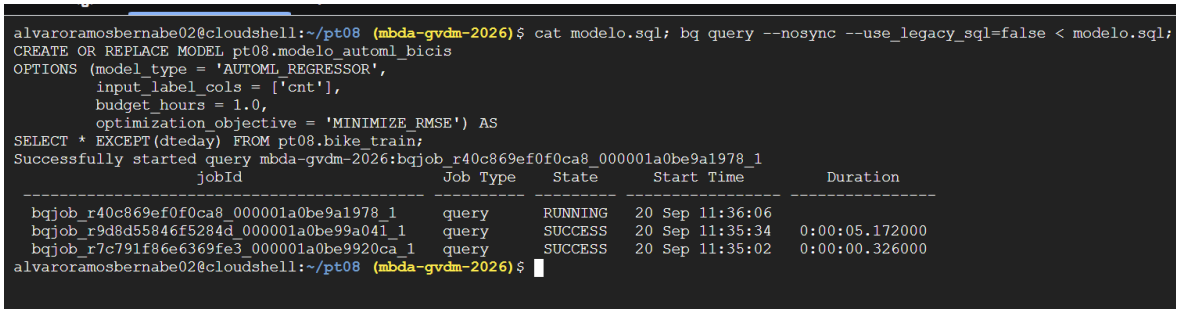


Figura 8. Lanzamiento del modelo `pt08.modelo_automl_bicis` (AUTOML_REGRESSOR, presupuesto de 1 hora, objetivo MINIMIZE_RMSE) de forma asíncrona con `--nosync`.

Resultado del modelo

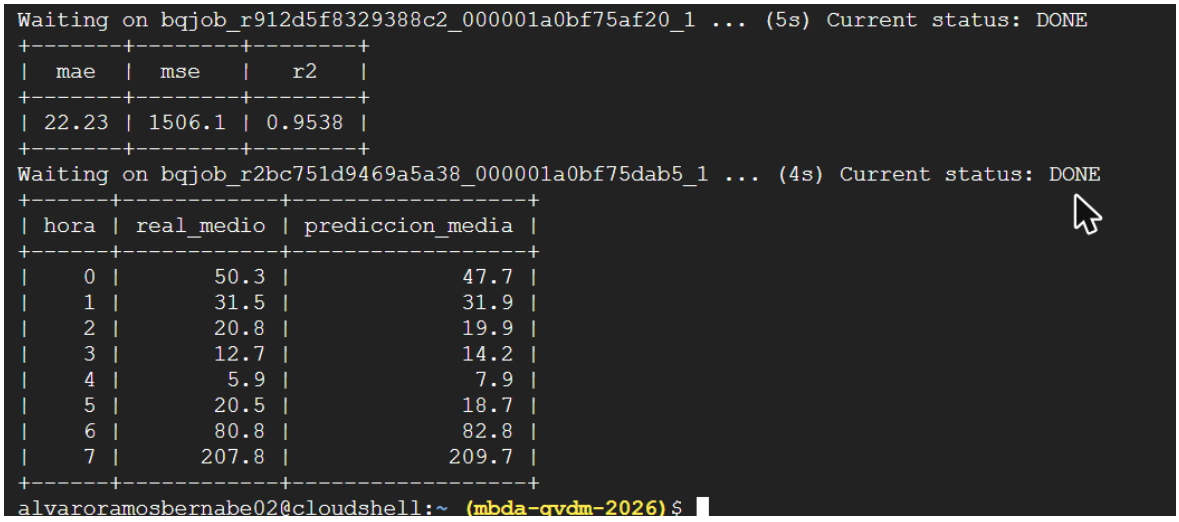


Figura 9. Evaluación del modelo sobre `bike_test` (datos no vistos) y comparación entre la demanda real y la predicha por horas.

Métrica	Valor	Interpretación
MAE (error absoluto medio)	22,23 bicicletas	El error típico de una predicción horaria es de unas 22 bicicletas.
MSE (error cuadrático medio)	1.506,1	RMSE ≈ 38,8 bicicletas: penaliza los errores grandes, concentrados en las horas punta.
R^2 (coeficiente de determinación)	0,9538	El modelo explica el 95,4 % de la varianza de la demanda horaria.

Qué significa el output del modelo automático

El modelo que devuelve BigQuery ML no es una fórmula que se pueda leer, sino un **conjunto de modelos seleccionado y ajustado automáticamente** por AutoML: el servicio ha probado distintas familias de algoritmos y configuraciones de hiperparámetros durante el presupuesto de una hora, se ha quedado con la combinación que minimiza el RMSE y la ha guardado

como un objeto que se consulta con `ML.PREDICT` igual que cualquier otro modelo. Lo que el usuario obtiene, por tanto, son **métricas de calidad y predicciones**, no coeficientes.

Leídas esas métricas: un **R^2 de 0,954** indica que el modelo reproduce casi por completo el patrón de la demanda, lo cual es coherente con este dataset, donde el número de alquileres depende de forma muy regular de la hora del día, el día de la semana y la meteorología. El **MAE de 22 bicicletas** hay que interpretarlo en relación con la magnitud de la serie: la demanda media ronda las 190 bicicletas/hora, con valores que van de menos de 10 en la madrugada a más de 900 en la hora punta, de modo que el error representa aproximadamente un 12 % de la media y es proporcionalmente mucho menor en las horas de mayor volumen. Que el **RMSE (38,8) sea casi el doble del MAE** revela que los errores no se reparten de forma uniforme: hay unas pocas horas —las puntas y los días con meteorología extrema— donde el modelo se desvía bastante más, mientras que en la mayoría de las horas acierta con holgura.

La comparación por horas de la captura lo confirma: la media real y la media predicha van prácticamente de la mano (50,3 frente a 47,7 a medianoche; 207,8 frente a 209,7 a las 7 de la mañana), es decir, el modelo ha aprendido la forma del perfil diario, con el valle nocturno y el pico del desplazamiento al trabajo.

Valoración del enfoque. AutoML resuelve en una consulta SQL lo que manualmente exigiría ingeniería de variables, selección de algoritmo y búsqueda de hiperparámetros, y por eso es una referencia excelente para saber rápidamente **qué rendimiento es alcanzable** con unos datos. A cambio, se pierde interpretabilidad —no se conoce el modelo elegido ni se controla su complejidad—, el coste es sensiblemente mayor que el de un modelo `BOOSTED_TREE` o `LINEAR_REG` de BigQuery ML (se factura por hora de entrenamiento en Vertex AI) y se queda corto cuando el negocio exige explicar cada predicción. En un proyecto real, lo razonable es usar AutoML como línea de referencia y valorar después si un modelo propio, más barato y explicable, alcanza un rendimiento parecido.

Eliminación de los recursos creados

Conforme al aviso del enunciado, se eliminan la función, los dos buckets, el conjunto de datos de BigQuery con sus tablas y el modelo, y el repositorio `gcf-artifacts` que Cloud Functions crea para almacenar las imágenes de las compilaciones:

```
gcloud functions delete borrar-fichero --region=$REGION --gen2 --quiet
gcloud storage rm --recursive gs://$BUCKET --quiet
gcloud storage rm --recursive gs://$DATOS --quiet
bq rm -r -f -d $PROJECT:pt08
gcloud artifacts repositories delete gcf-artifacts --location=$REGION --quiet
```



Figura 10. Eliminación de la función, los buckets, el conjunto de datos y el repositorio de imágenes.

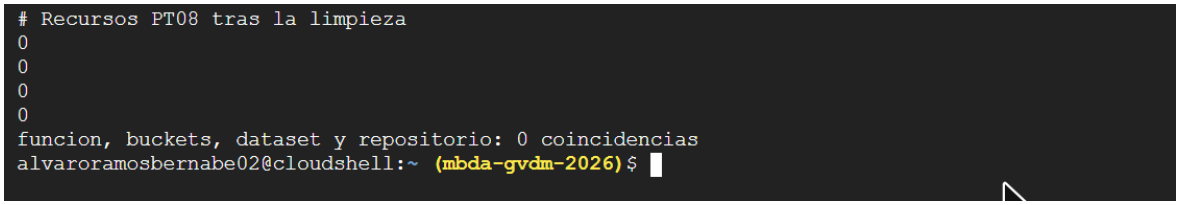


Figura 11. Comprobación posterior: 0 coincidencias para la función, los buckets, el conjunto de datos y el repositorio.

Referencias

Bike Sharing Dataset (UCI): https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset

BigQuery ML, modelos AutoML:
https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-automl

Cloud Run Functions con disparador de Cloud Storage:
https://docs.cloud.google.com/functions/docs/samples/functions-cloudevent-storage

Eventarc y agentes de servicio: https://cloud.google.com/eventarc/docs/roles-permissions